

# Switch (de)bouncing



R. Ferrero, P. Bernardi

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>



# Switch bounce

- When you push a button, press a micro switch or flip a toggle switch, two metal parts come together.
- For the user, it might seem that the contact is made instantly.
- In reality, the switch is bouncing between in-contact, and not in-contact.



# Why a switch bounces

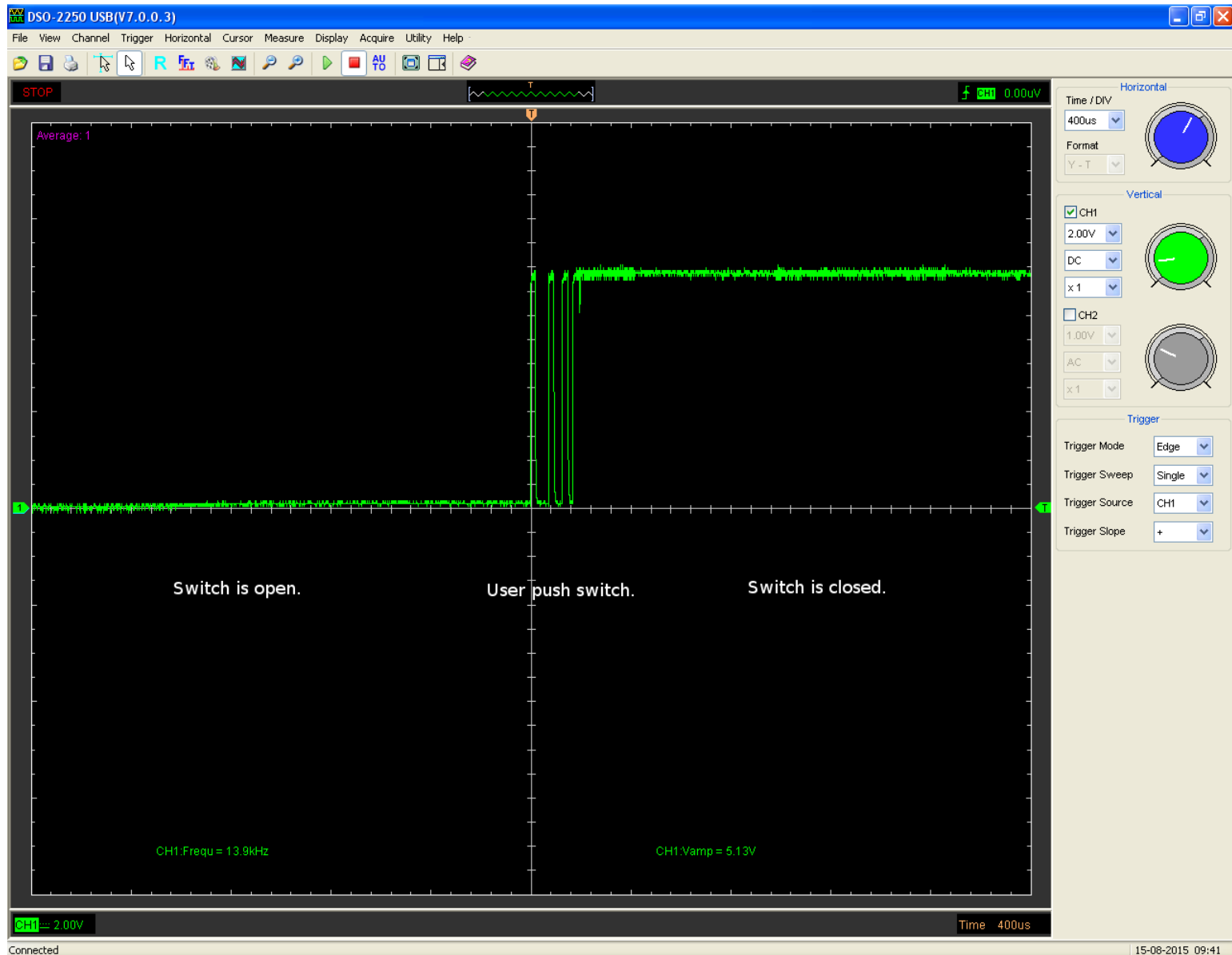
- Inside the switch there are moving parts.
- When you push the switch, it initially makes contact with the other metal part, but just in a brief split of a microsecond.
- Then it makes contact a little longer, and then again a little longer.
- In the end the switch is fully closed.
- A nice experiment with 4 switch components:

<https://www.allaboutcircuits.com/technical-articles/switch-bounce-how-to-deal-with-it/>

# Bouncing measurement

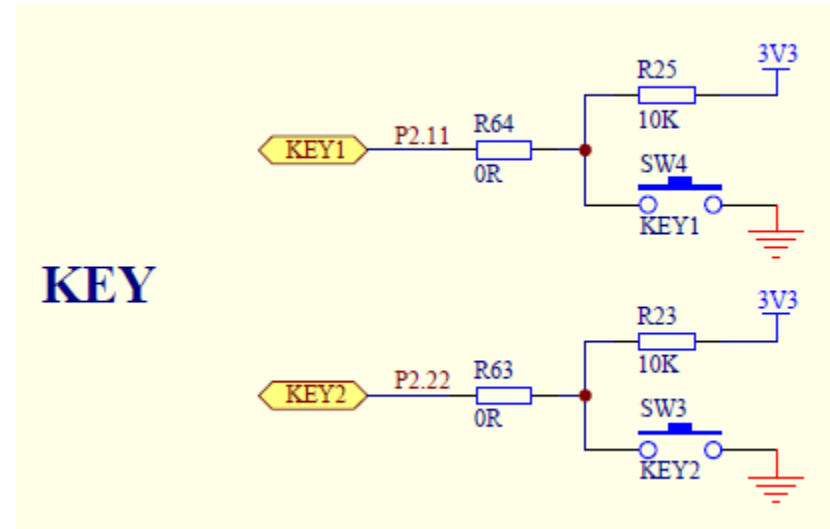
- "When the switch is closed, the two contacts actually separate and reconnect, typically 10 to 100 times over a period of about 1ms."  
["The Art of electronics", Horowitz & Hill, second edition, page 506]
- The SoC works faster than the bouncing, which results in that the hardware thinks you are pressing the switch several times.

# Switch bounce measurement



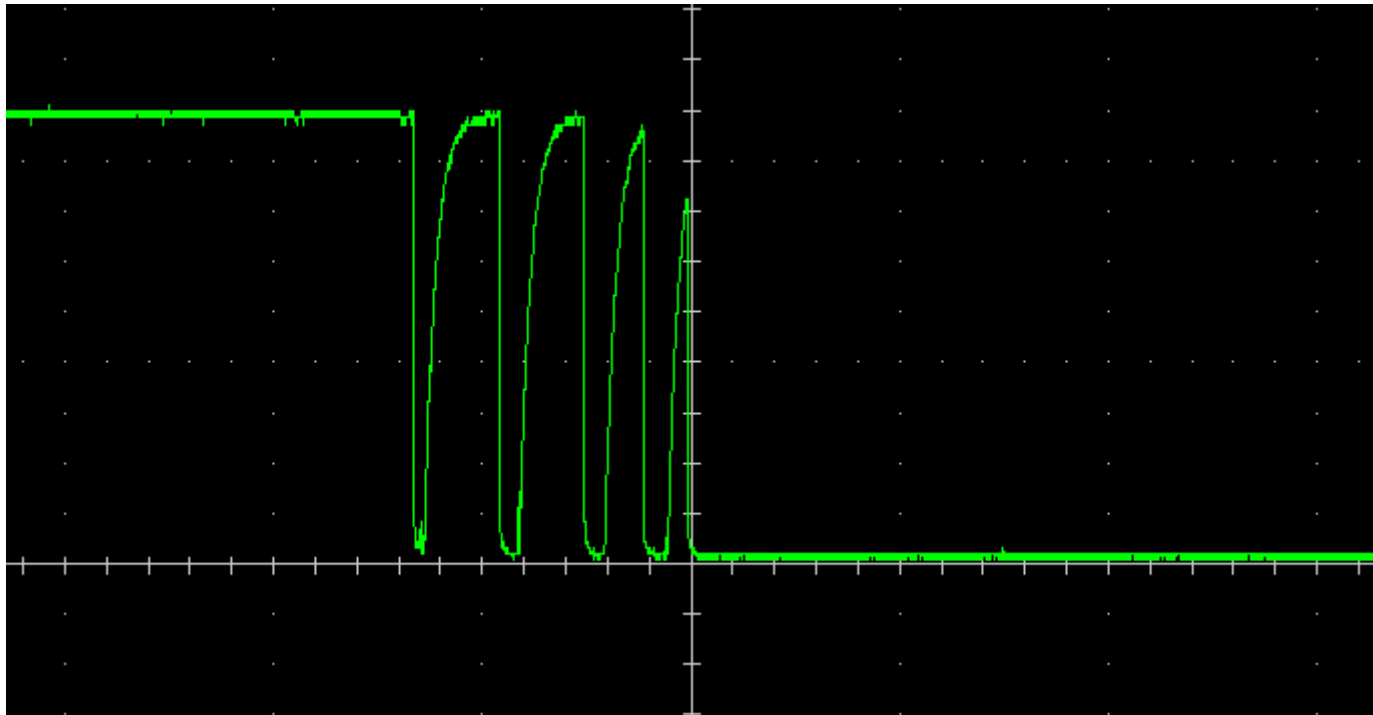
# Key behavior

- When pressed, the button drives the logic value 0
- When released, the button drives the logic value 1



# Button pulse-low pressure

Released

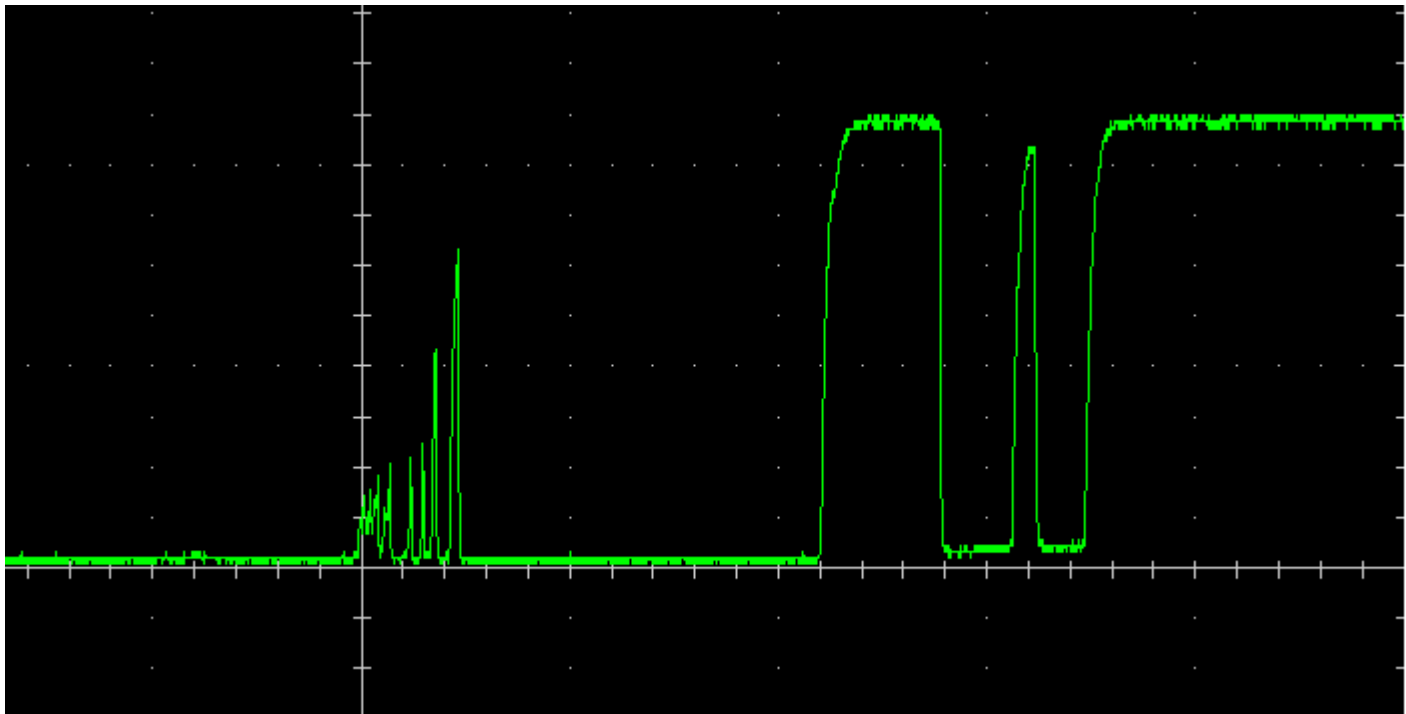


Pressed

# Button release

- Spikes may also show up and be triggered during pressure of the button.

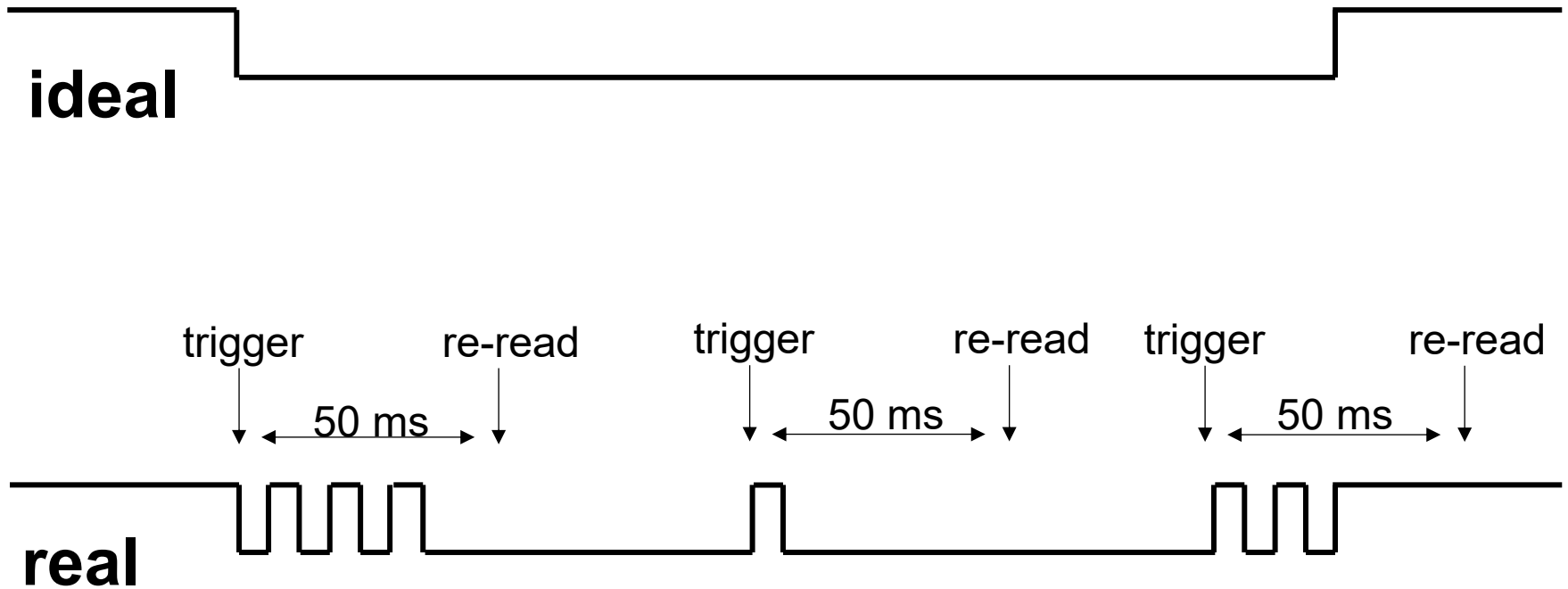
Released



Pressed

# Software debounce

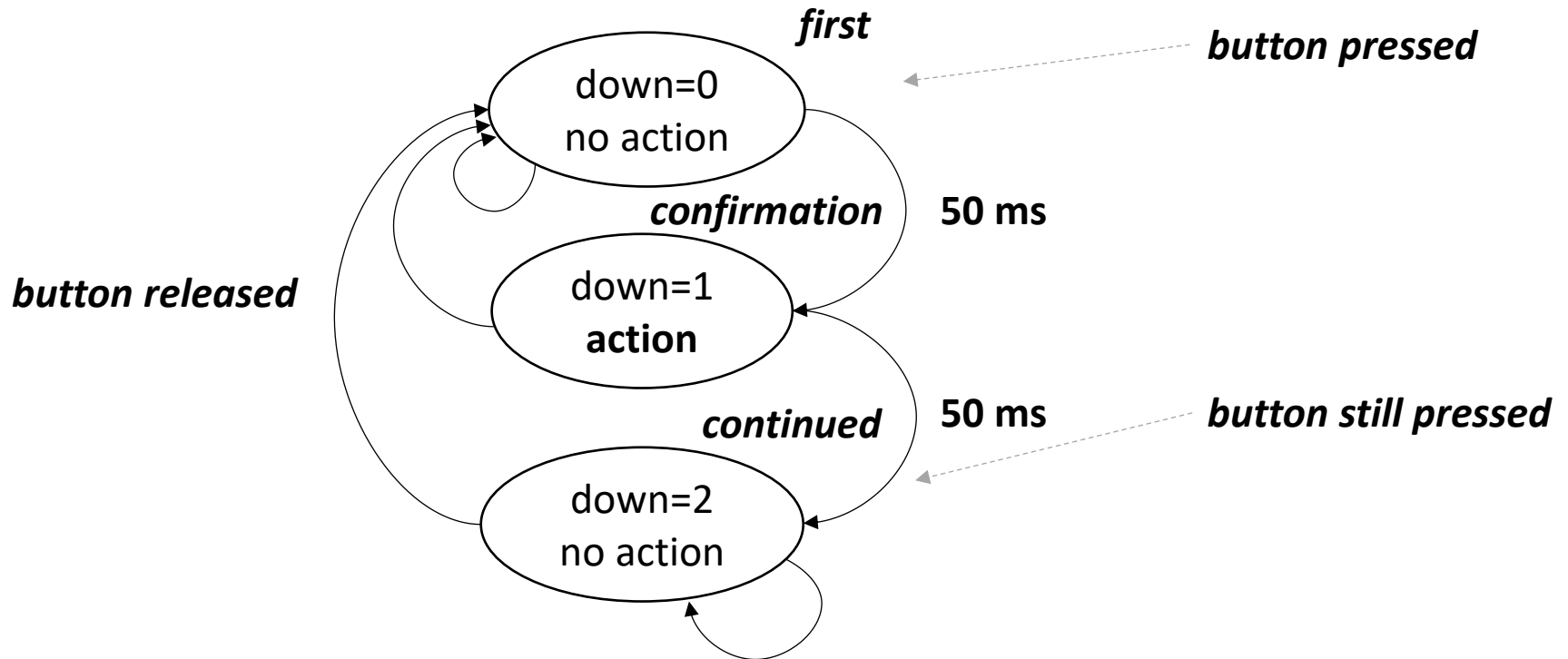
- The value of the pin is re-read after 50ms delay from the first bounce.



# Implementation

- Interrupt mode, if available (e.g., for buttons)
  - Power efficiency: the system can enter power down mode
- Polling the value of button related pins, if interrupt is not available
  - A timer can be used to wake up the system at regular time
- A blocking delay is deprecated
  - “for/while/do-while” empty constructs keeps the microcontroller occupied

# Finite state machine



- 50 ms (at least) is a reasonable human touch duration

# Exercise

- Use the external interrupt functionalities to mitigate Key bouncing
- Advanced: implement a "timer controlled polling strategy" to mitigate Joystick bouncing
- Quite advanced: manage the pressure of many buttons and the contemporary use of buttons and Joystick.
- Super advanced: manage joystick and implement button debouncing with RIT only.